hhu

hhu
Heinrich Heine
Universität Düsseldorf

HeineOS

Assignment material: Memory

Prof. Dr. Michael Schöttner
Dr. Fabian Ruhland

Aspects of Memory Management

Heinrich Heine Universität Düsseldorf

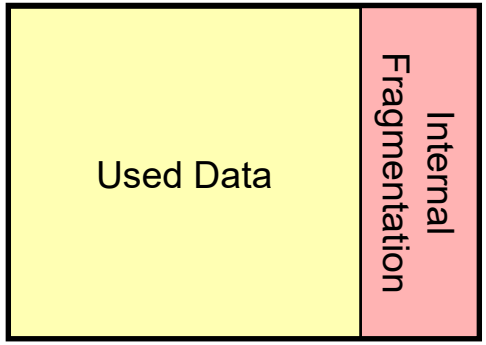
- Wasted memory (internal and external fragmentation)
- Managing free/occupied memory
- Reintegration of unused (freed) memory

Internal Fragmentation

- **If more memory is supplied than was requested, unused memory is wasted**
  - On the other hand, sparing very small chunks of memory makes no sense
    → These small heap-chunks need to be managed and are almost unusable

- This is especially critical, if many small chunks of memory are allocated
  - Example: 28 Byte are allocated,
    32 Byte are supplied → 4 Byte are wasted
  - If this happens 1 billion times,
    we lose 4 GB of main memory

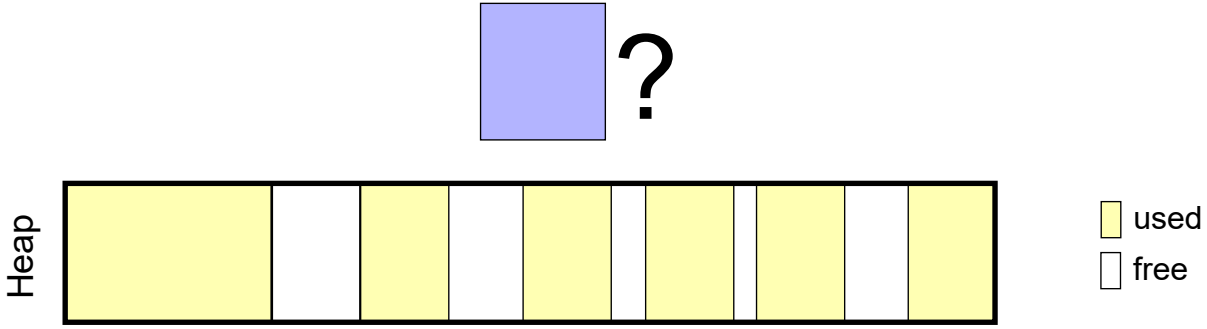
Chunk of memory on the heap



Internal Fragmentation

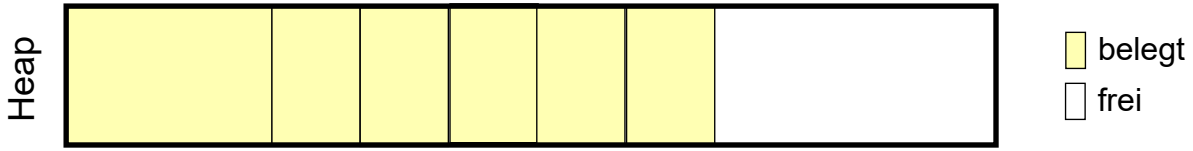
- Over time, allocations and deallocations result in many small chunks of free memory scattered throughout the heap

- The heap may not be able to serve a large allocation request, although in theory enough free memory is available (but not **coherent**)



Internal Fragmentation

- **Compacting** the heap solves this problem



- This is time-consuming for many blocks and may require hardware support

Managing free memory in a table

- Free chunks of memory are managed in a separate table
- Example (f=free, u=used, Unit: KB): 1f, 5u, 4f, 9u, 6f, 3u, 5f

Size	Adress
1	0
4	16
5	28
6	19

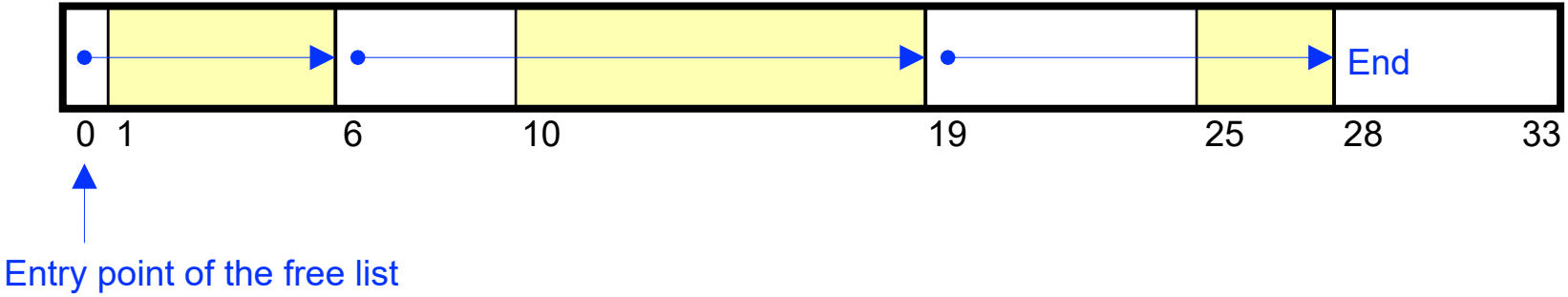
Managing free memory in a table

Heinrich Heine Universität Düsseldorf

- Advantages:
  - Easy to implement
  - Table can be stored in a separate memory area, protecting it from unintentionally being overwritten by the program
- Disadvantages:
  - Memory demand for the table is unclear
    - Depends on the amount of free memory chunks → Unkown at compile time
  - An allocation of a free memory chunk results in a gap in the table
    → The table memory also needs to be managed

Managing free memory in a list

- Link free chunks of memory through pointers
- Example (f=free, u=used, Unit KB): 1f, 5u, 4f, 9u, 6f, 3u, 5f



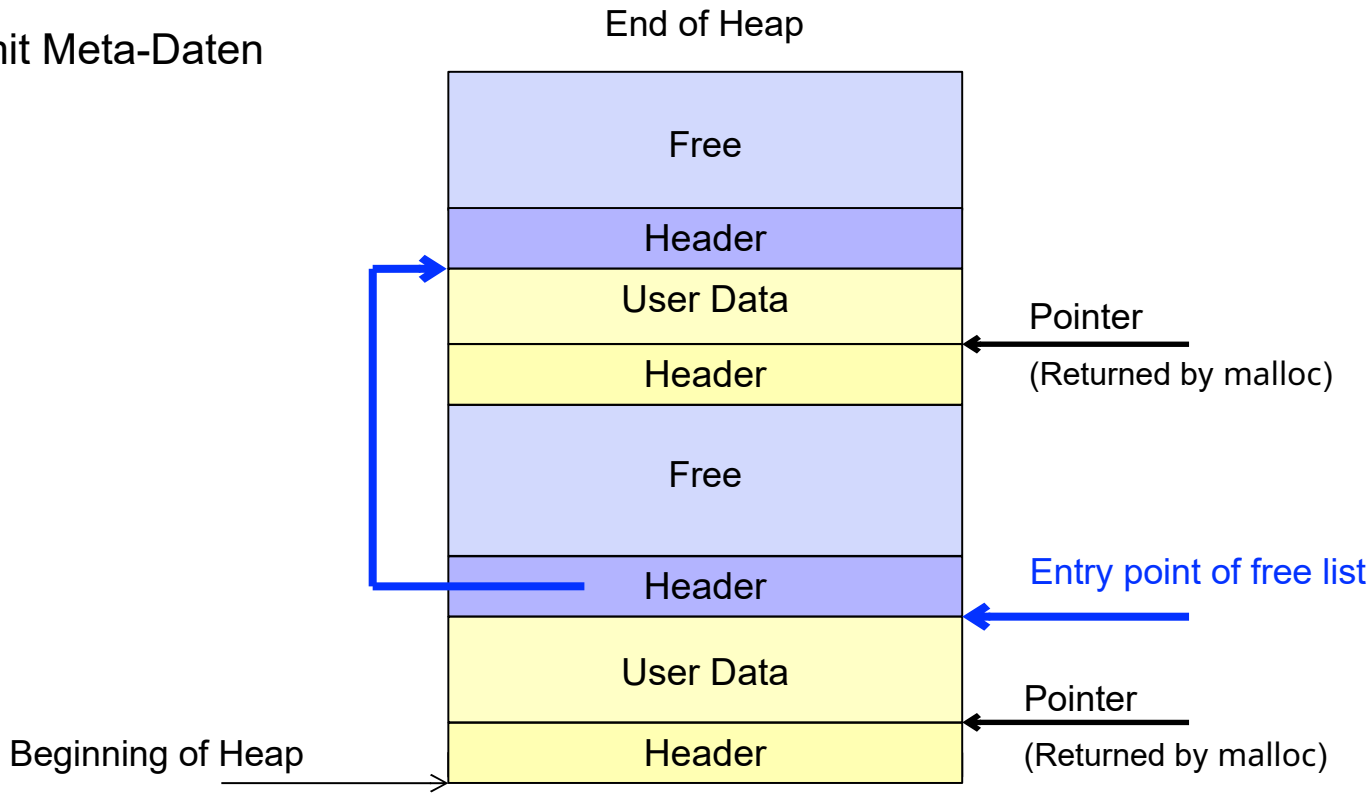
Managing free memory in a list

Heinrich Heine Universität Düsseldorf

- Advantages:
  - Does not require additional memory, as the free heap memory itself is used for storing the pointers and size information
- Disadvantages:
  - If a program writes beyond the boundary of an allocated chunk, it may corrupt the list
  - Sequential search for a fitting block is slow
- Optimization
  - Use multiple lists for different magnitudes of chunk sizes
    - Each list requires its own entry point

Managing free memory in a list

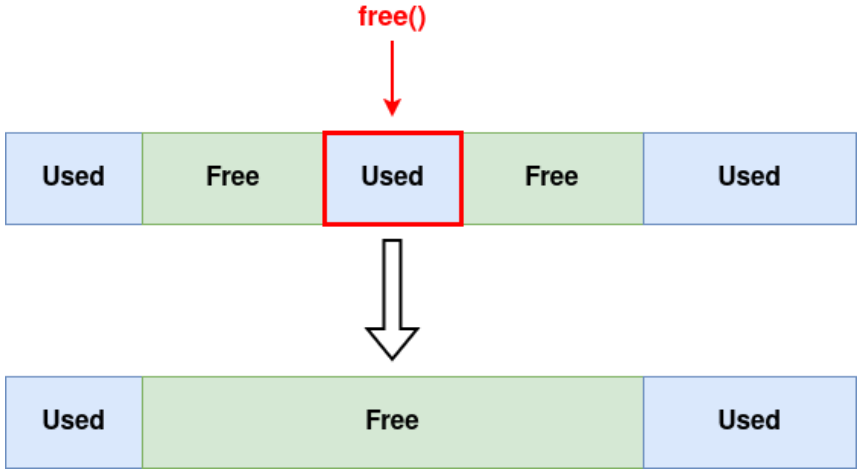
- Detail-Ansicht mit Meta-Daten



Reintegration of freed memory

- When a chunk of memory is freed, check if other free chunks are immediately adjacent
  → If so, merge adjacent chunks

- This prevents fragmentation of the heap into many small chunks



Memory Management in Rust

Heinrich Heine Universität Düsseldorf

- We create the heap directly after the kernel code/data (see `consts.rs`)
  - This is technically unsafe, as the physical memory at this location may be occupied by devices, the UEFI BIOS or bootloader data
    → A sophisticated allocator for physical memory is implemented in the second course
- Our memory manager needs to implement the `GlobalAlloc` trait:
  - `unsafe fn alloc(&self, layout: Layout) -> *mut u8`
  - `unsafe fn dealloc(&self, ptr: *mut u8, layout: Layout)`
- More information is provided in the code; Philipp Oppermann has written a nice article around mememory management in Rust: https://os.phil-opp.com/heap-allocation/